Migrating Scientific Calculator on ATtiny-85 from ATmega-328p

EE25BTECH11030-Josyula G S Avaneesh EE25BTECH11054-Harsha Soma Vardhan

Introduction:
This report details the architectural decisions, hardware selection, and algorithmic optimizations required to implement a fully functional order-of-operations (Shunting Yard) scientific calculator on an ATtiny85 microcontroller. Constrained by 6,012 bytes of usable Flash memory and lacking native floating-point hardware, the project required transitioning to a custom 32-bit fixed-point mathematical engine, eliminating standard libraries, and aggressively optimizing UI elements to prevent memory overflow.

1 Hardware Architecture and Cost Analysis

1.1 Bill of Materials

The hardware was selected to balance cost, form factor, and input/output capabilities. The primary challenge was interfacing a 25-button matrix and a 16x2 LCD using the highly limited GPIO pins of the ATtiny85. The components were sourced using standard Indian electronics market pricing.

TABLE I: Bill of Materials and Project Cost (in INR)

Component	Purpose	Qty	Est. Cost
Digispark (ATtiny85)	Core MCU (16MHz, 8KB Flash)	1	230Rs
16x2 LCD (with I2C)	Visual Display (reduces pin usage to 2)	1	200Rs
74HC595,74HC165 Shift Registers	Matrix column/row scanning expansion	1 each	20Rs
Tactile Push Buttons	5x5 Keypad Matrix	25	50Rs
220Ω, 10kΩ Resistors	Current limiting / Short-circuit protection	1, 5	10Rs
100nF Capacitors	IC Decoupling & Hardware Debouncing	3	10Rs
(Breadboard, wires)	Physical prototyping	1	90Rs
Total			**610Rs**

1.2 Cost Comparison: ATtiny85 vs. ATmega328P

An obvious architectural alternative to the ATtiny85 would have been an ATmega328P. The ATmega328P provides 32KB of Flash memory and sufficient native GPIO pins to drive the matrix directly without shift registers.

However, opting for an Atmega328p would increase the core microcontroller cost from 250Rs to approximately 400Rs - 500Rs. While the ATmega328P easily accommodates bloated standard C `<math.h>` floating-point libraries, it removes the strict memory constraints that forced the creative algorithmic problem-solving at the heart of this project, allowing it to also include functions like exponential and logarithmic, along with being able to use fundamental algorithms like RK4. The ATtiny85 was deliberately selected to keep the budget highly optimized while functioning as close to an actual scientific calculator as possible.

Upon calculating the total cost while using an Atmega328p instead of an ATtiny85, we find that the total is **800Rs**. This increase in price is due to the Atmega328p not having a built-in oscillator and being unable to regulate power.

2 Circuit Operation and Component Justification

2.1 System Operation Overview

The ATtiny85 has only 6 standard GPIO pins. One is tied to the Reset function, leaving 5 usable pins. To drive both an LCD and a 25-button matrix, extreme multiplexing is required:

- **Pins PB0 & PB2:** Dedicated to the I2C protocol (SDA/SCL) to communicate with the 16x2 LCD module.
- **Pins PB1, PB3, PB4:** Dedicated to a serial Shift Register protocol (Clock, Latch, Data) to sequentially scan the 5x5 keypad matrix. The Data pin (PB4) acts dynamically as both an output to push row states and an input to read column states.

2.2 Justification of Passive Components

- 10kΩ **Resistors:** Placed on the column lines of the button matrix. When pressing multiple buttons simultaneously in a standard matrix, a direct short circuit can occur between a HIGH output pin and a LOW output pin, potentially frying the microcontroller or the shift register. The 10kΩ resistors safely limit this current to roughly 0.5mA at 5V, protecting the ICs.
- 100**nF Ceramic Capacitors:** Used for logic-level decoupling. High-speed switching inside the ATtiny85 and the 74HC595 shift register causes micro-voltage dips on the power rail. Placing a 100nF capacitor physically close to the V_{CC} and GND pins of these ICs acts as a local energy reserve, smoothing out the voltage and preventing sudden microcontroller resets.
- 220Ω **Shared Bus Hack:** To drive both the 74HC595 and 74HC165 shift registers using a single data pin (`PB4`), the chips must share a data bus, which introduces a critical hardware conflict known as bus contention,*a conflict occurring when two or more devices or signals attempt to drive the same bus or node simultaneously, causing unpredictable outputs, increased power consumption, or potential hardware damage*.
  Because the 74HC165's output pin (`Q7`) is always actively driving the line HIGH or LOW, a catastrophic dead short would occur if the ATtiny85 simultaneously output an opposing logic state during a write cycle to the 74HC595, easily destroying both ICs. Placing a 220Ω resistor inline between `PB4` and `Q7` perfectly solves this by acting as a collision buffer. When a logic mismatch occurs (creating a 5V difference across the resistor), Ohm's Law

TABLE II: Hardware Wiring Connections

Source Component	Source Pin	Target Component	Target Pin	Notes
Power & Decoupling				
Power Supply	5V	Global VCC Rail	All 5V Pins	Main logic power
Power Supply	GND	Global GND Rail	All GND Pins	Common ground
100nF Capacitor	Leg 1	74HC595 & 165	VCC (Pin 16)	Place near ICs
100nF Capacitor	Leg 2	74HC595 & 165	GND (Pin 8)	Decoupling
I2C LCD Display				
Digispark	PB0 (SDA)	I2C LCD Module	SDA	I2C Data
Digispark	PB2 (SCL)	I2C LCD Module	SCL	I2C Clock
I2C LCD Module	VCC	Power Rail	5V	LCD Power
I2C LCD Module	GND	Power Rail	GND	LCD Ground
Shift Register Control Bus				
Digispark	PB1 (CLK)	74HC595	SHCP (Pin 11)	Shared Clock
Digispark	PB1 (CLK)	74HC165	CP (Pin 2)	Shared Clock
Digispark	PB3 (LATCH)	74HC595	STCP (Pin 12)	Shared Latch
Digispark	PB3 (LATCH)	74HC165	PL (Pin 1)	Shared Latch
Data Bus & 220 Ohm Collision Protection				
Digispark	PB4 (DATA)	74HC595	DS (Pin 14)	Direct serial write
Digispark	PB4 (DATA)	220 Ohm Resistor	Leg 1	Short-circuit buffer
220 Ohm Resistor	Leg 2	74HC165	Q7/QH (Pin 9)	Protects PB4
74HC595 (Row Driver - Output)				
74HC595	Q0 - Q4	Keypad Matrix	Rows 1 - 5	Drives rows LOW
74HC595	OE (Pin 13)	Power Rail	GND	Output Enable
74HC595	MR (Pin 10)	Power Rail	5V	Master Reclear
74HC165 (Column Reader - Input)				
74HC165	D0 - D4	Keypad Matrix	Cols 1 - 5	Reads columns
74HC165	CE (Pin 15)	Power Rail	GND	Clock Enable
74HC165	SER (Pin 10)	Power Rail	GND	Serial Input (Tie low)
Matrix Pull-ups (10k Ohm)				
Keypad Matrix	Cols 1 - 5	10k Ohm Resistors	Leg 1	One per column
10k Ohm Resistors	Leg 2	Power Rail	5V	Pulls cols HIGH

($I = 5\text{V}/220\Omega$) dictates that the short-circuit current is safely throttled to approximately 22.7mA. This limit sits comfortably below the 40mA absolute maximum rating of the ATtiny85 and the ±25mA limit of the 74HC series, allowing the microcontroller to safely overpower the 165's signal to write data to the 595 while reading input flawlessly when the pin switches to a high-impedance state.

3 Hardware Wiring Connections

4 Software Hardships and Explicit Compromises

To fit a Shunting Yard mathematical evaluator into the remaining $\sim 6,000$ bytes of memory, severe software compromises were deliberately made:

1) **Elimination of Floating-Point Hardware:** The standard `<math.h>` library ballooned the code to $7,610$ bytes. It was entirely replaced by a custom 32-bit fixed-point engine scaled by $1,000$.
2) **Removal of Logarithms & Floating Exponents:** Calculating natural logs (ln) or exponents (e^x) using Taylor Series loops causes 32-bit integer overflow (exceeding 2.1 billion) when using a 1000-scale fixed-point system. These functions were abandoned in favor of integer-only powers.
3) **UI Translation Layer Stripped:** To save ~ 150 bytes of memory for the Modulus and Absolute Value functions,

the translation code that expanded inputs (e.g., converting 's' to "sin(") was deleted. The LCD now displays raw memory buffer inputs (e.g., `s30=`).

4) **Forced Degree Mode:** Fixed-point math loses extreme precision when handling irrational numbers like radians (π). Trigonometry was hardcoded to Degrees to maintain perfect whole-number inputs for the lookup tables.

5 Code Efficiency and Flash Memory Map

TABLE III: Estimated Flash Memory Allocation

Software Module	Approx. Flash Footprint (Bytes)
Shunting Yard Evaluator (Order of Operations)	$\sim 1,800$
Main Control Loop & Matrix Scanner	$\sim 1,500$
I2C Protocol & LCD Drivers	~ 600
Fixed-Point Math Engine (Integer scaling, bounds)	~ 450
Trigonometric `PROGMEM` Lookup Tables	224
Compiler Overhead & Vector Tables	$\sim 1,300$
Total Utilized Memory	**5,874 / 6,012 (97.7%)**

6 Mathematical Error Analysis: Fixed-Point vs. Continuous Algorithms

Because the calculator utilizes a 1000-scale fixed-point engine and lookup tables, specific mathematical tolerances and truncation errors exist. To properly contextualize these errors, this system's accuracy is benchmarked against the standard floating-point algorithms typically employed on larger microcontrollers (such as the Arduino Uno/ATmega328P), specifically the 4th-order Runge-Kutta (RK4) method and the Quake III inverse square root algorithm.

6.1 System-Specific Error Tolerances

- **General Truncation Error** Division operations are integer-based. Any decimal precision beyond the third decimal place (0.001) is truncated, not rounded. Maximum general precision error is -0.0009.
- **Forward Trigonometry (**sin**,** cos**,** tan**):** The system uses a 91-point integer lookup table mapped exactly to degrees (0° to 90°). Inputs are cast to integers before lookup. Therefore, sin(45.6°) is evaluated as sin(45°). The maximum angular resolution error is 0.99°.
- **Inverse Trigonometry (**arcsin**,** arccos**,** arctan**):** Implemented via a 21-point lookup table with linear interpolation. Because the curve of $\arcsin(x)$ steepens dramatically near $x = 1$, the linear interpolation introduces an error margin of approximately ±0.5° for values between bounds.
- **Square Root:** Utilizes an integer-based Newton-Raphson approximation algorithm. Convergence is strictly integer-bounded, yielding a maximum floor truncation error of -0.001.

6.2 Comparative Analysis: Table Lookups vs. ODE Solvers (RK4)

Standard academic implementations of scientific calculators on microcontrollers calculate trigonometric and exponential functions by reformulating them as Ordinary Differential Equations (ODEs).

- **Accuracy vs. Processing Cost:** The RK4 method achieves extreme floating-point accuracy, with its local truncation error bounded by $O(h^5)$, where h is the step size. However, calculating four distinct variables (k_1, k_2, k_3, k_4) per step requires intense floating-point multiplication. Our ATtiny85 system abandons this continuous accuracy in favor of $O(1)$ constant-time memory lookups, trading $O(h^5)$ precision for a hardcoded worst-case resolution error of 0.99°.
- **Root Approximations:** Standard implementations on larger chips like the ATmega328P use the famous "Quake III" algorithm to calculate roots. This algorithm uses a specific bit-level "magic constant" (`0x5f3759df`) to rapidly guess the answer, but it fundamentally requires the system to support complex floating-point decimal numbers. Because our ATtiny85 completely removes the heavy floating-point library to save over 1.5KB of Flash memory, we cannot use this trick. Instead, our system uses a simpler, whole-number (integer) guessing method. While this integer method introduces a tiny precision error (truncating by up to -0.001), it successfully calculates roots without the massive memory overhead required by standard decimal algorithms.

Ultimately, the algorithms utilized in systems like the ATmega328p provide superior mathematical continuity and precision. However, they demand memory capacities far exceeding the ATtiny85's 6KB limit. The discrete fixed-point tables and bounded integer Newton-Raphson methods deployed in this project represent the necessary compromises required to force advanced scientific evaluation onto heavily constrained 8-bit architecture.

7 Development and Debugging Code

During the development phase, isolated code snippets were used to verify hardware functionality before integrating the complex math engine.

7.1 LCD I2C "Blink" Debug Code

This code was used to verify I2C communication and LCD addressing (0x27) without the overhead of matrix scanning.

The code is present in the https://github.com/Avaneesh-Josyula/Scientific-calculator-on-ATtiny85/tree/main/codes/blink/src directory under the name *main.c*

7.2 Matrix Keypad Debug Code

This snippet was utilized to confirm that the shift register (74HC595) was correctly pulsing the rows and that the ATtiny85 was successfully reading the columns, outputting the raw grid coordinate to the LCD.

The code is present in the

https://github.com/Avaneesh-Josyula/Scientific-calculator-on-ATtiny85/tree/main/codes/button_matrix/src

directory under the name *main.c*

8 Final Optimized Firmware

The final production code incorporates the fixed-point math engine,the 91-point trigonometric lookup table, the 21-point inverse trigonometric lookup table, the complete Shunting Yard string parsing evaluator, and the dual-mode mapping system containing Modulo and Absolute Value functions.

The code is present in the

https://github.com/Avaneesh-Josyula/Scientific-calculator-on-ATtiny85/tree/main/codes/main/src

directory under the name *main.c*

9 Circuit

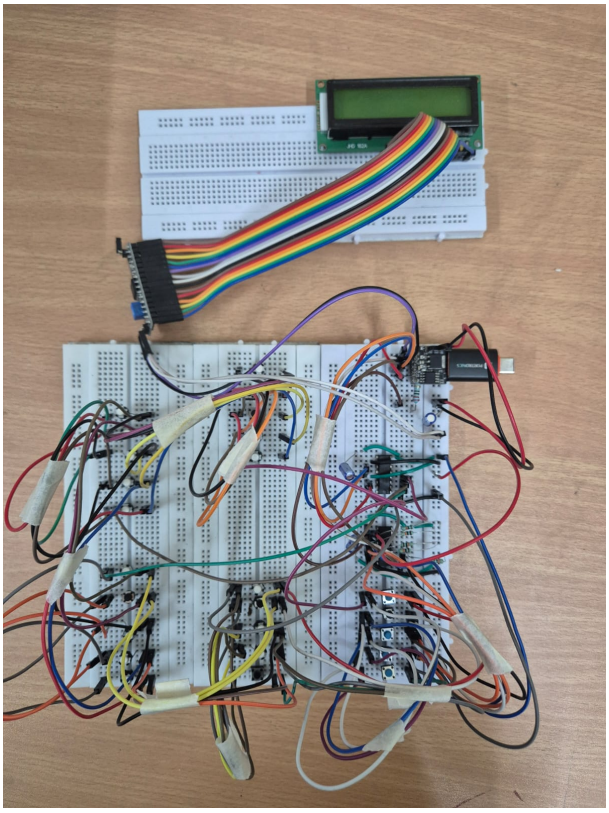

Fig. 1: Circuit picture